Journal of
Risk and Financial Management

MDPI

Article

A Comparative Study of Transformer-Based and Classical Models for Financial Time-Series Forecasting

Ting Liu

School of Computer Science, Northwestern University, Evanston, IL 60208, USA; tingliu2027@u.northwestern.edu

Abstract

This study compares classical and deep learning models (ARIMA, Random Forest, RNN, LSTM, CNN, and Transformer) for forecasting one-day-ahead log returns $r_{t+1} = \ln(P_{t+1}/P_t)$ using daily data for six U.S.-listed equities (NVDA, TSLA, SMCI, GOOGL, PYPL, SNAP) from 2014 to 2024. Predictors include lagged price/return information, lagged macroeconomic variables (CPI, policy rate, GDP) to reflect information availability, and technical indicators (SMA, RSI, MACD) computed using rolling windows ending at day t to avoid look-ahead bias. Performance is evaluated in a walk-forward out-of-sample design, with hyperparameters selected using time-series validation within each training window. Empirically, results are asset-dependent: ARIMA and Random Forest remain strong baselines; deep learning models show asset-dependent performance, with LSTM occasionally competitive in some settings, and the Transformer competitive but not uniformly dominant. For context, this study also reports a rule-based SMA(10/50) crossover benchmark evaluated net of transaction costs. Overall, the findings suggest that predictive signals in daily equity returns, when present, are modest and must be assessed under strict leakage controls and realistic evaluation protocols.

Keywords: transformer-based; deep learning; time-series forecasting; walk-forward validation; ARIMA; Random Forest

1. Introduction

Time-series forecasting is important in financial domains, supporting trend analysis, risk management, and modeling short-term return dynamics. Common forecasting baselines include ARIMA (H. Li & Liu, 2023), classical machine learning methods such as Random Forest (Tyralis & Papacharalampous, 2017), and deep learning models (e.g., RNN, LSTM, CNN) that have shown promise in capturing certain short-horizon patterns (Widiputra et al., 2021; Fischer & Krauss, 2018). Prior work has studied both price and return forecasting using deep learning approaches, including Transformer-based variants (Bhogade & Nithya, 2024; Gezici & Sefer, 2024; Zeng et al., 2023). This study focuses on one-day-ahead return forecasting; specifically, we predict the next-day log return $r_{t+1} = \ln(P_{t+1}/P_t)$ using information available up to day t.

From a financial economics perspective, return predictability is expected to be limited under the Efficient Market Hypothesis (Fama, 1970), and any exploitable structure is typically weak and unstable. Nevertheless, short-horizon predictability may arise due to time-varying risk premia, behavioral under/overreaction, market microstructure effects, and broader information frictions. The goal of this study is not to claim strong market inefficiency, but to provide a controlled comparison of forecasting models under strict out-of-sample evaluation.

Academic Editors: Florentin Serban and Thanasis Stengos

Received: 18 January 2026
Revised: 21 February 2026
Accepted: 3 March 2026
Published: 9 March 2026

Recent advances in deep learning, particularly Transformer architectures, provide a flexible way to model sequential data via self-attention. In financial return forecasting, however, reported gains can be sensitive to evaluation design and data leakage. Accordingly, the novelty of this study is methodological and empirical: we enforce strict leakage control and perform fold-local hyperparameter selection using only the validation window in each walk-forward split (the test window is never used for tuning). Under this stricter protocol, we find that classical baselines (ARIMA and Random Forest) remain difficult to beat and that deep models are not uniformly dominant, suggesting that some previously reported deep learning gains may be sensitive to evaluation design rather than reflecting robust predictive improvements. Accordingly, this study conducts a controlled comparison of classical and deep learning models—ARIMA, Random Forest, RNN, LSTM, CNN, and an encoder-only Transformer implementation—under a walk-forward out-of-sample protocol. All predictors are computed using information available up to time t, and preprocessing (e.g., scaling) and hyperparameter selection are performed within each training window using time-series validation to avoid look-ahead bias. Consistent with the view that return predictability is limited and unstable, this study emphasizes out-of-sample performance and interprets results as asset-dependent rather than universally dominated by any single model. Classical baselines such as ARIMA and Random Forest remain valuable due to their simplicity and interpretability (Tyralis & Papacharalampous, 2017; H. Li & Liu, 2023).

The main contributions of this paper are as follows:

- A leakage-aware walk-forward evaluation of classical and deep learning models for next-day log-return forecasting across six U.S.-listed equities (2014–2024).
- A feature construction pipeline that respects information timing (lagged macro variables; technical indicators computed with rolling windows ending at day t).
- A comparison that reports forecasting metrics and a simple rule-based SMA(10/50) benchmark net of transaction costs as a contextual reference.
- A transparent tuning protocol with a fixed search budget across models, where all hyperparameters (including ARIMA order and deep-model settings) are selected by minimum validation RMSE within each walk-forward fold; the search spaces are reported in Table 1 and the tuning budget is audited in Tables A1 and A2 in Appendix A.

Table 1. Hyperparameter search spaces and tuning protocol (validation-only within each walk-forward fold).

Model	Search Space/Selection Rule
ARIMA	(p,d,q) grid: $p \in \{0,1,2,3,4,5\}$, $d \in \{0,1\}$, $q \in \{0,1,2,3,4,5\}$; select by minimum validation RMSE (test never used).
Random Forest	Grid: `n_estimators` $\in \{200,500,1000\}$; `max_depth` $\in \{3,5,10,$`None`$\}$; `min_samples_leaf` $\in \{1,5,10\}$; select by minimum validation RMSE.
RNN/LSTM	Grid: hidden $\in \{32,64,128\}$; layers $\in \{1,2\}$; dropout $\in \{0,0.1,0.3\}$; lr $\in \{10^{-4}, 5\times10^{-4}, 10^{-3}\}$; batch $\in \{16,32,64\}$; early stopping on validation loss (patience = 3), select by minimum validation RMSE.
CNN	Grid: filters $\in \{16,32,64\}$; kernel $\in \{3,5\}$; dropout $\in \{0,0.1,0.3\}$; lr $\in \{10^{-4}, 5\times10^{-4}, 10^{-3}\}$; batch $\in \{16,32,64\}$; early stopping on validation loss (patience = 3), select by minimum validation RMSE.
Transformer	Grid: $d_{\text{model}} \in \{32,64,128\}$; heads $\in \{2,4,8\}$; layers $\in \{1,2,3\}$; dropout $\in \{0,0.1,0.3\}$; lr $\in \{10^{-4}, 5\times10^{-4}, 10^{-3}\}$; batch $\in \{16,32,64\}$; early stopping on validation loss (patience = 3), select by minimum validation RMSE.

Note: Hyperparameters are tuned within each walk-forward fold using the validation window only; the test window is evaluated once and never used for tuning or model selection.

2. Data Collection and Pre-Processing

2.1. Data Preparation

Data sources. This study obtains daily adjusted close prices for U.S.-listed equities from the Yahoo Finance API for the period 2014–2024. Adjusted close prices are used to ensure consistency under dividends and stock splits. The set of six stocks analyzed is reported in Table 2. Macroeconomic series (CPI, policy rate, and GDP) are incorporated as predictors and aligned to reflect information availability (details below).

Table 2. Selected stocks.

Ticker	Company
NVDA	NVIDIA Corporation, California, CA, USA
TSLA	Tesla, Inc., California, CA, USA
SMCI	Super Micro Computer, Inc., California, CA, USA
GOOGL	Alphabet Inc., California, CA, USA
PYPL	PayPal Holdings, Inc., California, CA, USA
SNAP	Snap Inc., California, CA, USA

Prediction target. Let P_t denote the adjusted close price on trading day t. The next-day (one-step-ahead) log return is defined as

$$r_{t+1} = \ln\left(\frac{P_{t+1}}{P_t}\right), \tag{1}$$

and the task is to predict r_{t+1} using only information available up to (and including) day t.

Metric scale. All error metrics (MAE, MSE, RMSE, and Huber loss) are computed on the original return scale so that reported values are directly comparable across models within each asset.

Input construction. All models use the same 60-trading-day lookback. For each day t, the feature sequence is formed from $[t-59, \ldots, t]$, and the label is r_{t+1}. This uniform rolling-window construction ensures consistent comparisons across model classes.

Walk-forward evaluation and model selection. This paper adopts a walk-forward (rolling-origin) protocol to mitigate overly optimistic estimates. In each fold, models are trained on an earlier period, hyperparameters are selected using the subsequent validation window, and performance is reported on the next unseen test segment. Test segments are evaluated once per fold and are never used for model selection or tuning.

Walk-forward split. In each fold, this study trains on the most recent T_{train} observations, validates on the next T_{val} observations, and tests on the subsequent T_{test} observations. The window then advances by S trading days and repeats until the end of the sample. Unless otherwise stated, we use $T_{\text{train}} = 1260$, $T_{\text{val}} = 252$, $T_{\text{test}} = 63$, and $S = 63$.

Missing values, scaling, and leakage control. Missing values are handled using forward fill only; any remaining missing observations at the beginning of a series are removed. To avoid look-ahead bias, all data transformations are fit using the training split only within each fold (including imputation rules, feature scaling, and any feature transformations), and the learned parameters are then applied unchanged to the validation and test splits.

Technical indicators and macro alignment. All technical indicators are computed using rolling windows ending at day t (no centered windows), and the label r_{t+1} is constructed strictly from P_{t+1} and P_t. Macroeconomic variables are aligned using an "as-of" information set with conservative publication lags (e.g., CPI available with approximately a one-month lag and GDP with approximately a one-quarter lag) so that predictors reflect information plausibly available at time t. Monthly and quarterly macro values are mapped

to daily trading days by carrying forward the most recently available released value until the next release. We use revised macroeconomic series (final-vintage values, as commonly provided by FRED); as a limitation, real-time vintage data are not used, and conservative release lags are applied to reduce revision-related leakage.

Reproducibility and implementation details. All experiments are implemented in Python 3.11. Classical baselines use `statsmodels` 0.14 and `scikit-learn` 1.4, and deep learning models are implemented in TensorFlow (Keras API; TensorFlow 2.16.x). Unless otherwise stated, this study fixes a global random seed (42) and applies it consistently across libraries to control stochastic components (data splits, initialization, and minibatch ordering). This study develops and runs preprocessing locally on macOS 14.3 (Apple M2, 32 GB RAM), and trains deep learning models on Google Colab using an NVIDIA A100 40 GB GPU. Hyperparameters are selected only using the validation window within each walk-forward fold; the test window is never used for model selection or tuning. The full hyperparameter search spaces and tuning protocol are reported in Section 2.2 and Table 1.

2.2. Experimental Setup

This study uses a leakage-safe walk-forward evaluation. Within each fold, hyperparameters are selected using only the validation window; the test window is never used for tuning or model selection. This study also uses a fixed hyperparameter search budget defined by the candidate grids in Table 1. Specifically, the number of evaluated configurations per ticker per fold is 72 (ARIMA), 36 (Random Forest), 162 (RNN/LSTM), 162 (CNN), and 729 (Transformer). In all cases, the selected configuration is the one with the lowest validation RMSE. For ARIMA, the (p, d, q) order search is treated as the ARIMA counterpart of hyperparameter tuning and follows the same validation-only selection rule. The test window is evaluated once and never used for tuning or model selection.

For transparency, Tables A1 and A2 in Appendix A report the tuning budget (number of evaluated configurations) and the decomposition (cardinality) of the candidate hyperparameter grids under the validation-only selection rule used in each walk-forward fold (the test window is never used for tuning/model selection).

2.3. Macroeconomic Indicators

To incorporate market-wide information, we merge three macroeconomic series from the Federal Reserve Economic Data (FRED) database:

- Consumer Price Index (CPI; `CPIAUCSL`) as an inflation proxy;
- Short-term policy interest rate (Federal Funds Rate; `FEDFUNDS`);
- Gross Domestic Product (real GDP; `GDPC1`) as an economic activity proxy.

Macroeconomic releases arrive with reporting delays. To approximate real-time availability and avoid look-ahead bias, we apply a publication-lag convention: CPI is shifted by one month and GDP by one quarter, so that the value used at day t reflects information that would plausibly have been available by t. The policy-rate series is available at a lower frequency and is mapped to the daily calendar by forward-filling the most recently released value.

To integrate monthly/quarterly series with daily equity data, macro variables are mapped to the daily calendar using forward fill only (no backward fill). Consequently, on each day t, macro predictors depend only on the most recently available released observation.

2.4. Feature Engineering

This study constructs technical indicators commonly used to summarize recent price dynamics.

To avoid look-ahead bias, all indicators are computed using trailing rolling windows that end at day t (i.e., no centered windows), and the resulting values are used to predict the next-day return r_{t+1}. Specifically, we compute the following:

- **SMA(50):** 50-day simple moving average of the adjusted close, computed from $\{P_{t-49}, \ldots, P_t\}$.
- **RSI(14):** 14-day Relative Strength Index computed from trailing gains and losses over $\{t-13, \ldots, t\}$.
- **MACD(12,26,9):** MACD line defined as $\mathrm{EMA}_{12}(P_t) - \mathrm{EMA}_{26}(P_t)$, with a 9-day EMA signal line (all computed using information up to day t).

These features provide compact summaries of recent trends and momentum that may complement lagged returns in the forecasting models. All technical indicators are computed from price information available up to and including day t, with no centered windows, and are used to predict r_{t+1}.

Macroeconomic predictors are aligned to the daily calendar using an "as-of" information set with publication lags; details are provided in Section 2.3. This study uses revised FRED series (final-vintage values) rather than real-time vintage data (ALFRED), which we treat as a limitation.

3. Transformer-Based Model for Forecasting

This paper implements an encoder-only Transformer to forecast the next-day (one-step-ahead) log return $r_{t+1} = \ln(P_{t+1}/P_t)$ using information available up to (and including) day t. For each trading day t, we form an input sequence $X_t \in \mathbb{R}^{L\times d}$ consisting of the past L trading days of features,

$$X_t = \left[x_{t-L+1}, x_{t-L+2}, \ldots, x_t\right]^\top, \tag{2}$$

where each feature vector $x_\tau \in \mathbb{R}^d$ concatenates lagged price/return features, lagged macroeconomic predictors (aligned with publication lags), and technical indicators computed using rolling windows ending at day τ (no centered windows), ensuring leakage-safe inputs.

3.1. Input Embedding and Positional Encoding

Each time step is projected to a d_{model}-dimensional embedding via a learned linear map,

$$E_t = X_t W_E + \mathbf{1} b_E^\top, \qquad W_E \in \mathbb{R}^{d\times d_{\text{model}}}, \tag{3}$$

and combined with a positional encoding $P \in \mathbb{R}^{L\times d_{\text{model}}}$ to preserve temporal order,

$$Z_t^{(0)} = E_t + P. \tag{4}$$

This study uses a learned (trainable) positional embedding added to the input embeddings.

3.2. Transformer Encoder and Prediction Head

N Transformer encoder layers process the embedded sequence. In layer ℓ, Multi-head Self-Attention (MSA) and a position-wise Feed-Forward Network (FFN) are applied with residual connections and layer normalization (Add & Norm):

$$\tilde{Z}_t^{(\ell)} = \mathrm{LN}\Big(Z_t^{(\ell-1)} + \mathrm{MSA}(Z_t^{(\ell-1)})\Big), \tag{5}$$

$$Z_t^{(\ell)} = \mathrm{LN}\Big(\tilde{Z}_t^{(\ell)} + \mathrm{FFN}(\tilde{Z}_t^{(\ell)})\Big), \tag{6}$$

for $\ell = 1, \ldots, N$. After the final encoder layer, we obtain a fixed-dimensional representation using *last-timestep pooling*,

$$h_t = Z_t^{(N)}[L] \in \mathbb{R}^{d_{\text{model}}}, \tag{7}$$

i.e., we take the representation corresponding to the most recent day t (the last token in the sequence). Finally, a linear prediction head maps h_t to the one-step-ahead forecast (Widiputra et al., 2021),

$$\hat{r}_{t+1} = w^\top h_t + b, \tag{8}$$

where $w \in \mathbb{R}^{d_{\text{model}}}$ and $b \in \mathbb{R}$ are learned parameters (Gezici & Sefer, 2024).

3.3. Training Objective

The Transformer is trained within each walk-forward fold using the training window only, with hyperparameters selected on the validation window. Unless otherwise stated, we minimize the Huber loss between $\hat{r}_{t+1}$ and r_{t+1}. The architecture of the Transformer model is presented in Figure 1.

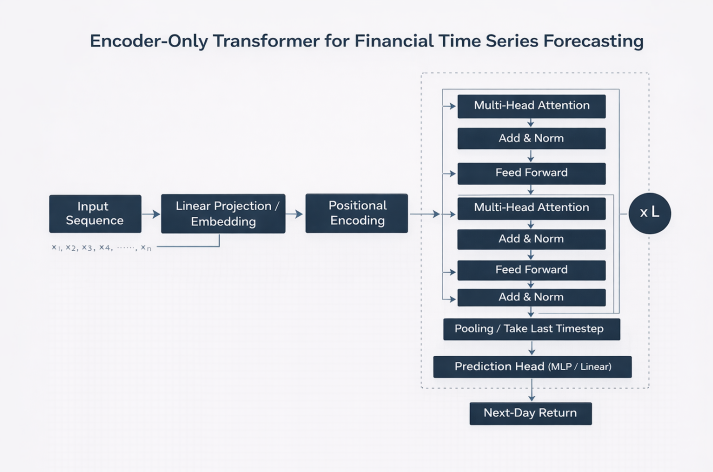


Figure 1. Encoder-only Transformer for Financial Time-Series Forecasting.

3.4. Data Preparation

This paper follows a walk-forward (rolling-origin) evaluation protocol to mimic real-time forecasting.

In each fold, the model is trained on a rolling historical window of fixed length T_{train}, hyperparameters are selected using the subsequent validation window, and performance is reported on the next unseen test window.

To avoid data leakage, feature scaling is fit using the training split only and then applied unchanged to the validation and test splits.

Inputs are constructed using a fixed lookback window of $L = 60$ trading days, so that the model receives the feature sequence $\{x_{t-L+1}, \ldots, x_t\}$ when predicting the next-day log return r_{t+1}.

3.5. Model Architecture

We use an encoder-only Transformer for one-step-ahead forecasting. Given an input feature sequence $X_t \in \mathbb{R}^{L \times d}$ constructed from information available up to day t, we first project each time step to a d_{model}-dimensional embedding via a learned linear map and add positional encoding to preserve temporal order. The resulting embedded sequence is processed by N Transformer encoder layers, each comprising multi-head self-attention and a position-wise feed-forward network with residual connections and layer normalization (Add & Norm). After the final encoder layer, we use last-timestep pooling, i.e., we take the

representation of the most recent time step (the last token) as a fixed-dimensional summary vector. A linear prediction head maps this vector to the one-step-ahead forecast $\hat{r}_{t+1}$.

3.6. Training and Optimization

Table 3 reports a representative Transformer configuration. In the main walk-forward evaluation, Transformer hyperparameters are selected within each fold using the validation window only, as described in Section 2.2 and Table 1 (the test window is never used for tuning). The model is trained with backpropagation using the Adam optimizer (learning rate 10^{-3}) to minimize the Huber loss with $\delta = 1.0$. We train for up to 10 epochs and apply early stopping based on validation loss with patience $= 3$. Performance is reported strictly out-of-sample under the walk-forward protocol.

Table 3. Transformer hyperparameters (encoder-only).

Parameter	Value
Model Dimension (d_{model})	64
Attention Heads (h)	4
Key Dimension per Head (d_k)	16
Encoder Layers (N)	2
Lookback Window (L)	60
Optimizer	Adam
Learning Rate	0.001
Loss Function	Huber ($\delta = 1.0$)
Dropout Rate	0 (selected; i.e., no dropout)
Max Epochs	10
Early stopping (patience)	3
Batch Size	16

Note: $d_{\text{model}} = h \cdot d_k$. Table 3 reports a representative configuration; hyperparameters are selected fold-locally as in Table 1.

4. Comparison Models

4.1. ARIMA (Autoregressive Integrated Moving Average)

The ARIMA model is a classical statistical approach for time-series forecasting that models a series using autoregressive and moving-average components (H. Li & Liu, 2023). In this study, ARIMA is used as a linear baseline for one-step-ahead forecasting under the same walk-forward evaluation protocol as the learning-based models. In each walk-forward fold, ARIMA is fit using the training window only, and the order (p, d, q) is selected via a grid search using the validation window (minimizing validation RMSE). This study searches $p \in \{0, 1, 2, 3, 4, 5\}$, $d \in \{0, 1\}$, and $q \in \{0, 1, 2, 3, 4, 5\}$ (Alamu & Siam, 2024). The selected model is then evaluated on the next unseen test segment; the test window is never used for tuning or model selection. While ARIMA can be effective when the series is approximately linear and stationary, it may have difficulty with the following:

- nonlinearities and volatility clustering in financial returns;
- longer-range dependencies beyond its chosen lag order;
- structural changes (time-varying dynamics) across market regimes.

For these reasons, ARIMA can be limited in highly nonlinear or regime-dependent settings, motivating comparisons with more flexible models (Kehinde et al., 2023; Tyralis & Papacharalampous, 2017).

4.2. LSTM (Long Short-Term Memory)

Long Short-Term Memory (LSTM) is a recurrent neural network variant designed to handle longer-range dependencies by using gated updates to an internal memory state (Kehinde et al., 2023; Tyralis & Papacharalampous, 2017). In our one-step-ahead forecasting setup, the LSTM takes the same L-day input feature window as the other learning-based models and outputs the next-day return forecast $\hat{r}_{t+1}$. Compared with a vanilla RNN, LSTM is typically more stable for sequential learning because the gating mechanism helps reduce vanishing-gradient issues and allows the model to retain relevant information across time (H. Li & Liu, 2023).

This study includes LSTM as a strong recurrent baseline and evaluates it under the same walk-forward out-of-sample protocol used for all models (Sunny et al., 2020).

4.3. RNN (Recurrent Neural Network)

A Recurrent Neural Network (RNN) models sequential data by updating a hidden state over time, so the prediction at day t can depend on information from earlier timesteps (Kehinde et al., 2023). RNNs can capture short-horizon dynamics, but in practice, they may struggle to learn long-range dependencies because gradients can vanish or explode during backpropagation through time (Zulqarnain et al., 2020). For this reason, RNNs are included as a baseline sequence model in our one-step-ahead forecasting setup, and their performance is compared with LSTM and Transformer models under the same walk-forward evaluation protocol.

4.4. CNN (Convolutional Neural Network)

Convolutional Neural Networks (CNNs) were first popular in image tasks, but they are also used for time-series forecasting by applying one-dimensional convolutions over the input sequence (Xu et al., 2023; H. Li & Liu, 2023). In this paper, the CNN uses the same L-day feature window as the other learning-based models. Convolution filters scan across the window to capture local patterns (short-horizon trend or momentum signals). The extracted features are then pooled and passed to a fully connected layer to produce the one-step-ahead forecast $\hat{r}_{t+1}$.

A limitation is that CNNs mainly focus on local information, so they may not capture very long-range dependencies as well as models designed for long memory. For this reason, this study includes CNN as a strong nonlinear baseline and evaluates it under the same walk-forward out-of-sample protocol (W. Li & Law, 2024).

4.5. Random Forest

Random Forest (RF) is an ensemble method that combines many decision trees using bagging and feature subsampling to produce a nonlinear prediction. In this study, RF is used as a classical machine-learning baseline for one-step-ahead return forecasting under the same walk-forward evaluation protocol as the other models. The model takes the tabular feature vector at day t (lagged returns/prices, macro variables, and technical indicators) and outputs the next-day forecast $\hat{r}_{t+1}$.

While RF can model nonlinear relationships and is relatively robust to overfitting, it does not explicitly model sequential dependence in the input window and relies on engineered lag features to capture time dynamics. This paper includes RF as a strong nonlinear baseline and compares it with the sequence models under identical data-leakage controls.

5. Model Evaluation and Performance Analysis

5.1. Error Analysis

This study evaluates one-step-ahead forecasting performance using four standard error metrics: mean absolute error (MAE), mean squared error (MSE), root mean squared error (RMSE), and Huber loss. Table 4 summarizes the metric values for each ticker–model pair, and Figures 2 and 3 visualize cross-model comparisons.

To ensure a fair comparison, all deep learning models (CNN, LSTM, RNN, and Transformer) were trained under the same leakage-safe walk-forward evaluation protocol and hyperparameters were selected using validation windows (see Section 2.2 for details). We used early stopping and fixed random seeds to reduce overfitting and improve reproducibility.

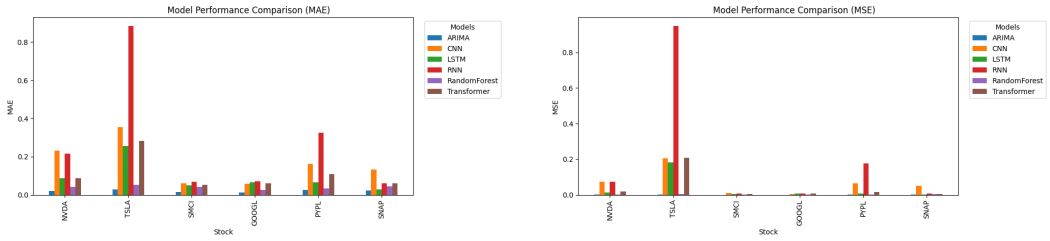
Table 4. Error analysis (Huber, MAE, MSE, RMSE) by stock and model.

Stock	Model	Huber	MAE	MSE	RMSE
GOOGL	ARIMA	0.000131	0.011833	0.000263	0.016202
GOOGL	CNN	0.002094	0.057280	0.004187	0.064708
GOOGL	LSTM	0.003538	0.066024	0.007075	0.084114
GOOGL	RNN	0.004559	0.070944	0.009118	0.095486
GOOGL	RandomForest	0.000491	0.027137	0.000981	0.031328
GOOGL	Transformer	0.003505	0.061703	0.007010	0.083728
NVDA	ARIMA	0.000335	0.019488	0.000669	0.025874
NVDA	CNN	0.037359	0.232881	0.074717	0.273344
NVDA	LSTM	0.006591	0.087193	0.013182	0.114814
NVDA	RNN	0.037315	0.215479	0.074629	0.273184
NVDA	RandomForest	0.001203	0.042178	0.002407	0.049058
NVDA	Transformer	0.009002	0.087683	0.018004	0.134178
PYPL	ARIMA	0.000618	0.025103	0.001236	0.035163
PYPL	CNN	0.032029	0.162290	0.064058	0.253097
PYPL	LSTM	0.003227	0.067139	0.006455	0.080341
PYPL	RNN	0.088035	0.325262	0.176160	0.419715
PYPL	RandomForest	0.001184	0.034084	0.002369	0.048669
PYPL	Transformer	0.008718	0.108786	0.017437	0.132049
SMCI	ARIMA	0.000300	0.016225	0.000600	0.024488
SMCI	CNN	0.005180	0.061963	0.010360	0.101784
SMCI	LSTM	0.002008	0.051037	0.004016	0.063369
SMCI	RNN	0.003722	0.067456	0.007444	0.086278
SMCI	RandomForest	0.001654	0.043116	0.003307	0.057507
SMCI	Transformer	0.001969	0.053435	0.003937	0.062746
SNAP	ARIMA	0.000949	0.024465	0.001897	0.043556
SNAP	CNN	0.024802	0.133296	0.049605	0.222722
SNAP	LSTM	0.001103	0.029171	0.002205	0.046960
SNAP	RNN	0.003288	0.061114	0.006577	0.081097
SNAP	RandomForest	0.002139	0.043993	0.004277	0.065400
SNAP	Transformer	0.002864	0.061151	0.005727	0.075677
TSLA	ARIMA	0.000856	0.029449	0.001712	0.041381
TSLA	CNN	0.102500	0.355587	0.206653	0.454591
TSLA	LSTM	0.091144	0.256797	0.182694	0.427427
TSLA	RNN	0.457674	0.884034	0.948471	0.973895
TSLA	RandomForest	0.002338	0.051717	0.004676	0.068380
TSLA	Transformer	0.103461	0.282166	0.207227	0.455222

A clear pattern is that ARIMA and Random Forest are difficult to beat in this dataset; this negative result is itself informative. Under leakage-safe walk-forward evaluation with *fold-local, validation-only* hyperparameter selection (the test window is never used for tuning) and a controlled tuning budget across models (Table 1 and Appendix A, Tables A1 and A2), the apparent dominance of high-capacity architectures becomes substantially weaker and

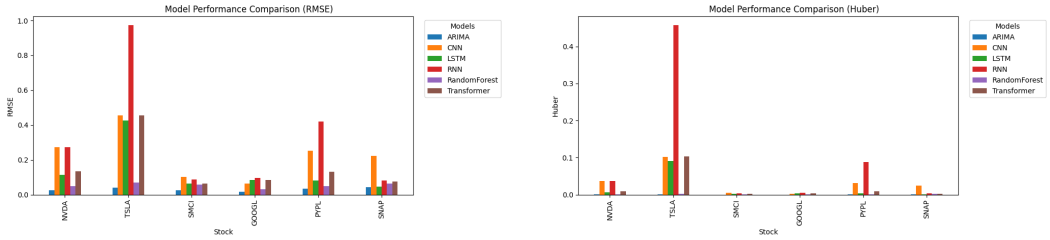
strongly asset-dependent. ARIMA yields the lowest errors for GOOGL, NVDA, PYPL, SMCI, and SNAP across multiple metrics; for TSLA, ARIMA is also best, while Random Forest remains a strong baseline. In contrast, the neural models do not show a consistent advantage over the classical baselines; for some tickers, they perform substantially worse. The Transformer remains competitive in specific cases (e.g., closer to Random Forest on SMCI) but does not consistently outperform ARIMA or Random Forest across the full set of stocks. Overall, Figures 2 and 3 suggest that model rankings are ticker-dependent, reinforcing the need for asset-specific modeling choices rather than a single globally dominant architecture.

From an economic perspective, this outcome is plausible because short-horizon equity return prediction is often characterized by a low signal-to-noise ratio and time-varying regimes. Simpler models can generalize well when the predictive structure is close to linear or short-memory, while high-capacity neural models may be more sensitive to limited effective sample size, volatility clustering, and regime shifts. Random Forest remains competitive on TSLA, which may reflect higher nonlinearity and volatility-driven dynamics in that series. Section 6.1 evaluates an exploratory forecast-to-trade mapping and highlights the importance of drawdown-aware risk controls (Tyralis & Papacharalampous, 2017; Xu et al., 2023).



(**a**) MAE comparison. (**b**) MSE comparison.

Figure 2. Model performance comparison (MAE and MSE).



(**a**) RMSE comparison. (**b**) Huber loss comparison.

Figure 3. Model performance comparison (RMSE and Huber).

5.2. Model Correlation Heatmap

Figure 4 reports the inter-model correlation of error patterns. For each model, we form a single error vector by concatenating its forecasting errors across all evaluated tickers and metrics (MAE, MSE, RMSE, and Huber). To avoid scale effects across metrics, each metric dimension is standardized before correlation is computed, and we report Pearson correlation (results are qualitatively similar using Spearman rank correlation).

The heatmap highlights several notable relationships. ARIMA and Random Forest exhibit a strong positive correlation (0.97), indicating that their *relative* error patterns across assets/metrics are closely aligned. CNN and LSTM are also strongly correlated (0.87), suggesting similar behavior under the current experimental setup. LSTM and Transformer show near-perfect correlation (0.99), implying that these two architectures tend to rank assets/metrics similarly in terms of forecasting error. RNN shows mixed behavior: it correlates strongly with the other neural models (e.g., 0.89 with LSTM and

0.91 with Transformer) but is notably weaker with Random Forest (0.36) and lower with ARIMA (0.40).

Overall, Figure 4 suggests that some models cluster into groups with similar *error patterns* (e.g., ARIMA–Random Forest and LSTM–Transformer). Importantly, correlation captures similarity in patterns rather than absolute accuracy; therefore, highly correlated models may still differ substantially in their error magnitudes. These relationships may motivate future work on model selection, regime-dependent switching, or ensemble design.

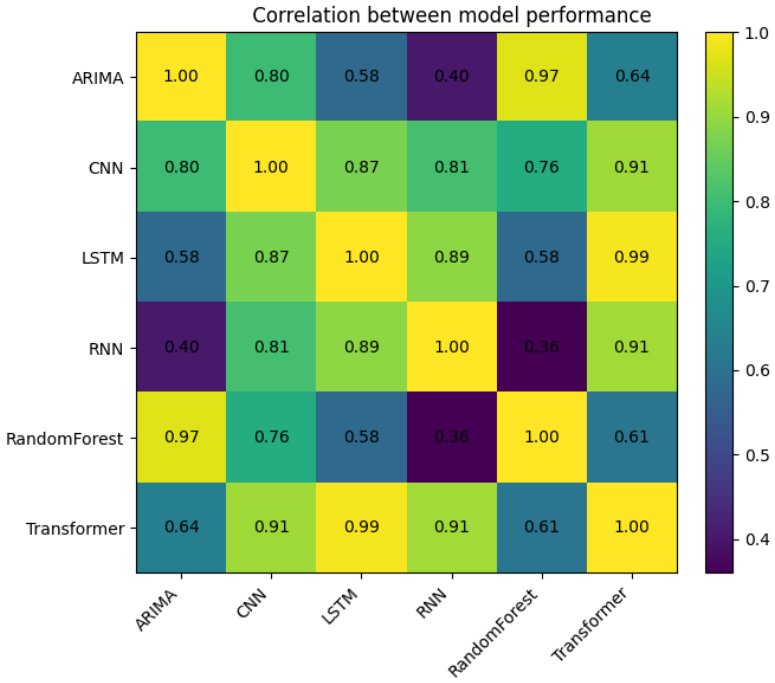


Figure 4. Heatmap: Correlation between model performance.

6. Backtesting Strategy and Trading Performance

6.1. Trading Strategy Design

As a simple benchmark, we use a moving-average crossover strategy with SMA(10) and SMA(50). The trading rule is as follows:

- **Long:** if $\text{SMA}_{10,t} > \text{SMA}_{50,t}$, hold a 100% long position.
- **Cash:** if $\text{SMA}_{10,t} \leq \text{SMA}_{50,t}$, hold cash (0% equity).

This study includes this SMA crossover as a standard trend-following baseline and compares its performance with SPY (adjusted close; price return), used as an investable proxy for the S&P 500.

Forecast-linked strategy. To examine how one-step-ahead return forecasts translate into trading decisions, we construct a simple *forecast-to-trade* mapping and apply it to the Transformer forecasts as a representative deep model. The purpose is methodological rather than to claim that any single forecasting model is universally best: forecast accuracy (e.g., RMSE) does not necessarily translate into trading profitability once the decision rule, turnover, and transaction costs are considered (Atsalakis & Valavanis, 2009). We therefore benchmark the forecast-linked strategy against a standard technical rule (SMA10/50) and buy-and-hold (SPY). For clarity, this study reports the trading results for a single representative instantiation of the forecast-linked rule (using Transformer forecasts) and benchmark it against SMA10/50 and buy-and-hold (SPY) under identical execution and transaction-cost assumptions. A systematic cross-model trading comparison (applying the

same rule to ARIMA and Random Forest forecasts) is left to future work to keep the trading analysis focused on the decision rule rather than a cross-model performance contest.

Let $\hat{r}_{t+1|t}$ denote the one-day-ahead return forecast formed at day t. The position weight is

$$w_t = \mathbb{I}\{\hat{r}_{t+1|t} > 0\}, \tag{9}$$

so the strategy is fully invested when the forecast is positive and otherwise remains in cash. To reduce excessive turnover from noisy daily signals, trades are executed only when the sign of the signal changes. We report performance both before and after transaction costs (Fischer & Krauss, 2018).

6.2. Backtest Setup

The price sample spans 2014–2024 and uses daily adjusted close prices. For trading and performance evaluation, simple (arithmetic) returns are computed as

$$R_{t+1} = \frac{P_{t+1}}{P_t} - 1$$

(using adjusted closes to account for splits/dividends).

To avoid look-ahead bias, signals are formed using information available up to day t, and trades are executed on the next trading day (returns realized over $t \to t+1$). Transaction costs are modeled as a one-way cost of $c = 5$ bps per trade (equivalently 10 bps per round-trip) and applied whenever the position changes:

$$R^{\text{net}}_{t+1} = w_t R_{t+1} - c\,|w_t - w_{t-1}|.$$

The forecasting target is the log return

$$r_{t+1} = \ln\left(\frac{P_{t+1}}{P_t}\right).$$

For the backtest, portfolio performance is computed using simple returns R_{t+1}, where

$$R_{t+1} = e^{r_{t+1}} - 1.$$

Trading positions are formed from the sign of the predicted log return $\hat{r}_{t+1}$, which is equivalent to using the sign of $\hat{R}_{t+1}$.

6.3. Performance Metrics

Table 5 reports net trading performance for the forecast-linked Transformer strategy, the SMA10/50 baseline, and the SPY benchmark. We evaluate performance using annualized return statistics, risk-adjusted ratios, and drawdown measures.

The annualized arithmetic mean return is computed from daily net simple returns assuming 252 trading days:

$$\mu_{\text{ann}} = 252\,\overline{R^{\text{net}}}, \tag{10}$$

where $\overline{R^{\text{net}}}$ is the sample mean of daily net simple returns R^{net}_t.

CAGR is computed from the equity curve as

$$\text{CAGR} = \left(\frac{V_{\text{end}}}{V_{\text{start}}}\right)^{252/N} - 1, \tag{11}$$

where N is the number of trading days and

$$V_t = \prod_{s \le t} (1 + R_s^{\text{net}}), \qquad V_{\text{start}} = 1.$$

Final wealth ($\times$) is defined as $W = V_{\text{end}}$. Sharpe and Sortino ratios are annualized from daily net returns assuming 252 trading days, with the risk-free rate set to 0.

Table 5. Trading performance comparison (net of transaction costs).

Metric	Transformer Strategy (Net)	SMA10/50 Strategy (Net)	SPY Benchmark
Annualized mean return (%)	12.13	21.00	15.76
CAGR (%)	4.97	19.95	14.91
Final wealth ($\times$)	1.20	1.98	1.68
Sharpe ratio	0.32	0.94	0.92
Sortino ratio	0.38	1.30	1.29
Maximum drawdown (%)	−65.69	−27.15	−24.50

Notes: Performance is reported net of transaction costs (10 bps per round-trip, implemented as 5 bps per one-way trade) and assumes 252 trading days per year with a zero risk-free rate for Sharpe and Sortino.

6.4. Stock-Level Trading Performance

Figure 5 compares the cumulative equity curves of the forecast-linked Transformer strategy, the SMA10/50 baseline, and the SPY benchmark (all net of transaction costs). The SMA strategy exhibits the strongest overall growth across the sample, while the Transformer-based strategy follows a substantially more unstable path, including a pronounced mid-sample drawdown before partially recovering toward the end. This contrast suggests that, under the current setup, the SMA baseline provides a more reliable trend-following profile, whereas the forecast-linked strategy is more sensitive to adverse regimes and therefore requires stronger drawdown and turnover control.

Figure 6 breaks down performance by ticker for the SMA10/50 strategy using an aligned evaluation window (i.e., all per-ticker curves are reported over the same start/end dates for comparability). The results show substantial cross-sectional dispersion: a small subset of stocks contributes most of the gains, with NVDA and TSLA exhibiting particularly strong cumulative growth, while other tickers display flatter or more volatile trajectories.

Overall, strategy profitability is concentrated in trend-dominated assets and varies materially across stocks, reinforcing the importance of stock-level evaluation, risk-adjusted performance measures, and drawdown-aware portfolio construction.

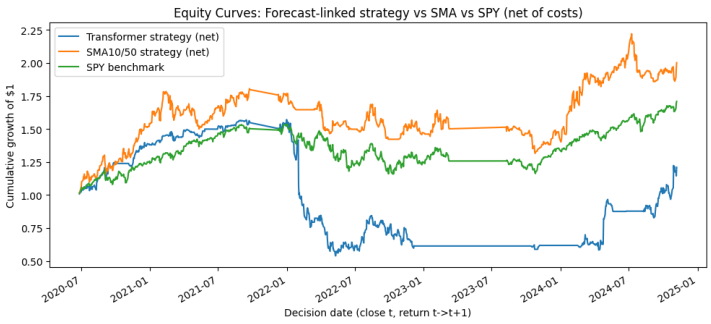


Figure 5. Equity curves: Transformer vs. SMA10/50 vs. SPY (net of costs).

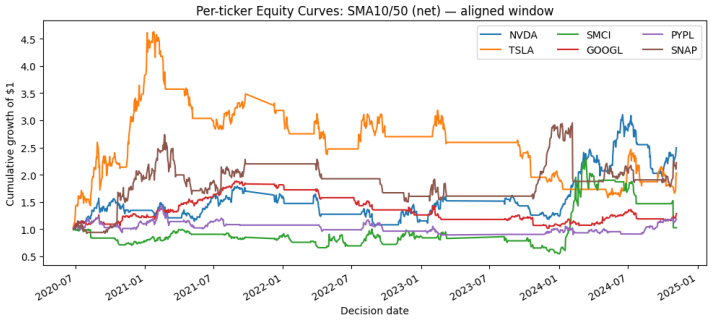


Figure 6. Per-ticker equity curves: SMA10/50 strategy (net), aligned window.

7. Limitations

A limitation of this study is that macroeconomic predictors are taken from revised FRED series rather than real-time vintages (ALFRED). Because data revisions can introduce an optimistic bias relative to what would have been available to an investor in real time, future work will rerun the pipeline using real-time vintages to further mitigate revision-related look-ahead effects. Future work will also test broader asset classes (e.g., ETFs, FX, and commodities) and higher-frequency data, and will evaluate alternative portfolio construction and risk-management rules beyond the simple long/cash mapping.

8. Conclusions and Future Work

This paper compares ARIMA, Random Forest, CNN/LSTM/RNN, and a Transformer for one-step-ahead return forecasting across six U.S.-listed equities (NVDA, TSLA, SMCI, GOOGL, PYPL, SNAP). The main takeaway is that model performance is highly ticker-dependent. In our results, ARIMA and Random Forest are difficult to beat for many assets, while neural models are less stable and can produce much larger errors. The Transformer is competitive in a few cases, but it does not consistently outperform simpler baselines across the full set of stocks.

This paper also evaluates a basic trading baseline (SMA10/50) alongside a forecast-linked Transformer strategy under the same backtesting assumptions (transaction costs and next-day execution). The SMA strategy performs well relative to SPY on a risk-adjusted basis in this sample. In contrast, the forecast-linked Transformer strategy exhibits substantially larger drawdowns, highlighting that forecast quality alone is insufficient and that the forecast-to-trade mapping is a first-order design choice.

Future work will focus on (i) refining the prediction target and horizon (e.g., direction/thresholded signals or volatility-aware targets), (ii) strengthening walk-forward validation with leakage-safe feature construction and hyperparameter tuning, and (iii) redesigning the trading layer with position sizing, turnover constraints, and drawdown-aware risk controls. It would also be useful to align macroeconomic variables by publication lag and, where possible, use real-time vintages so that inputs remain realistic in real time.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: Data were obtained from Yahoo Finance and FRED (Federal Reserve Economic Data). The datasets analyzed during the current study are publicly available from these sources.

Acknowledgments: The author used AI-assisted tools for language editing and formatting. The author reviewed and takes responsibility for the content.

Conflicts of Interest: The author declares no conflicts of interest.

Appendix A. Hyperparameter Tuning Evidence

For ARIMA, the (p, d, q) order is selected within each walk-forward fold using the validation window only, following the same validation-only rule as other models; the selected order minimizes validation RMSE (the test window is never used for tuning). Random Forest hyperparameters are selected by validation RMSE using the grid in Table 1. Neural models (RNN/LSTM/CNN/Transformer) are tuned using the validation window only (including dropout as in Table 1) with early stopping on validation loss (patience = 3) and restoring the best validation checkpoint within each fold.

Table A1. Tuning budget (grid size) and evaluation counts under validation-only selection within each walk-forward fold. The grid definitions follow Table 1; in all cases, the selected configuration minimizes validation RMSE, and the test window is never used for tuning/model selection.

Model	Configs Per Ticker Per Fold	Configs Per Fold (6 Tickers)	Selection Rule
ARIMA	72	432	Min validation RMSE
Random Forest	36	216	Min validation RMSE
RNN	162	972	Min validation RMSE (early stop on val loss)
LSTM	162	972	Min validation RMSE (early stop on val loss)
CNN	162	972	Min validation RMSE (early stop on val loss)
Transformer	729	4374	Min validation RMSE (early stop on val loss)
Total	1323	7938	–

Notes: The study uses six equities (NVDA, TSLA, SMCI, GOOGL, PYPL, SNAP) and a walk-forward protocol; tuning is performed inside each fold using the validation window only. RNN and LSTM share the same search space and tuning budget. Early stopping uses patience = 3 on validation loss (deep models), and the best configuration is selected by minimum validation RMSE.

Table A2. Decomposition of the hyperparameter grids (cardinality) reported in Table 1.

Model	Grid Components	Total Configs
ARIMA	$\lvert p\rvert = 6$ (0–5), $\lvert d\rvert = 2$ (0, 1), $\lvert q\rvert = 6$ (0–5)	$6 \times 2 \times 6 = 72$
Random Forest	$\lvert$`n_estimators`$\rvert = 3$, $\lvert$`max_depth`$\rvert = 4$, $\lvert$`min_samples_leaf`$\rvert = 3$	$3 \times 4 \times 3 = 36$
RNN/LSTM	$\lvert$hidden$\rvert = 3$, $\lvert$layers$\rvert = 2$, $\lvert$dropout$\rvert = 3$, $\lvert$lr$\rvert = 3$, $\lvert$batch$\rvert = 3$	$3 \times 2 \times 3 \times 3 \times 3 = 162$
CNN	$\lvert$filters$\rvert = 3$, $\lvert$kernel$\rvert = 2$, $\lvert$dropout$\rvert = 3$, $\lvert$lr$\rvert = 3$, $\lvert$batch$\rvert = 3$	$3 \times 2 \times 3 \times 3 \times 3 = 162$
Transformer	$\lvert d_{\text{model}}\rvert = 3$, $\lvert$heads$\rvert = 3$, $\lvert$layers$\rvert = 3$, $\lvert$dropout$\rvert = 3$, $\lvert$lr$\rvert = 3$, $\lvert$batch$\rvert = 3$	$3^6 = 729$

Notes: The grid values and the validation-only selection rule follow Table 1.

References

Alamu, O. S., & Siam, M. K. (2024). Stock price prediction and traditional models: An approach to achieve short-, medium-and long-term goals. *arXiv*, arXiv:2410.07220. [CrossRef]

Atsalakis, G. S., & Valavanis, K. P. (2009). Forecasting stock market short-term trends using a neuro-fuzzy based methodology. *Expert Systems with Applications*, *36*(7), 10696–10707. [CrossRef]

Bhogade, V., & Nithya, B. (2024). Time series forecasting using transformer neural network. *International Journal of Computers and Applications*, *46*(10), 880–888. [CrossRef]

Fama, E. F. (1970). Efficient capital markets: A review of theory and empirical work. *The Journal of Finance*, *25*(2), 383–417. [CrossRef]

Fischer, T., & Krauss, C. (2018). Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*, *270*(2), 654–669. [CrossRef]

Gezici, A. H. B., & Sefer, E. (2024). Deep transformer-based asset price and direction prediction. *IEEE Access*, *12*, 24164–24178. [CrossRef]

Kehinde, T. O., Khan, W. A., & Chung, S. H. (2023, October 10–12). *Financial market forecasting using RNN, LSTM, BiLSTM, GRU and transformer-based deep learning algorithms*. Proceedings of the IEOM International Conference on Smart Mobility and Vehicle Electrification, Detroit, MI, USA.

Li, H., & Liu, T. (2023). Portfolio optimization based on the LSTM forecasting model. *Proceedings of the 2nd International Conference on Financial Technology and Business Analysis*, *48*(1), 97–106. [CrossRef]

Li, W., & Law, K. E. (2024). Deep learning models for time series forecasting: A review. *IEEE Access*, *12*, 92306–92327. [CrossRef]

Sunny, M. A. I., Maswood, M. M. S., & Alharbi, A. G. (2020). Deep learning-based stock price prediction using LSTM and bi-directional LSTM model. In *Proceedings of the 2020 2nd novel intelligent and leading emerging sciences conference (NILES)* (pp. 87–92). IEEE .

Tyralis, H., & Papacharalampous, G. (2017). Variable selection in time series forecasting using random forests. *Algorithms*, *10*(4), 114. [CrossRef]

Widiputra, H., Mailangkay, A., & Gautama, E. (2021). Multivariate CNN-LSTM model for multiple parallel financial time-series prediction. *Complexity*, *2021*(1), 9903518. [CrossRef]

Xu, C., Li, J., Feng, B., & Lu, B. (2023). A financial time-series prediction model based on multiplex attention and linear transformer structure. *Applied Sciences*, *13*(8), 5175. [CrossRef]

Zeng, Z., Kaur, R., Siddagangappa, S., Rahimi, S., Balch, T., & Veloso, M. (2023). Financial time series forecasting using CNN and transformer. *arXiv*, arXiv:2304.04912. [CrossRef]

Zulqarnain, M., Ghazali, R., Ghouse, M. G., Hassim, Y. M. M., & Javid, I. (2020). Predicting financial prices of stock market using recurrent convolutional neural networks. *International Journal of Intelligent Systems and Applications*, *9*(6), 21. [CrossRef]